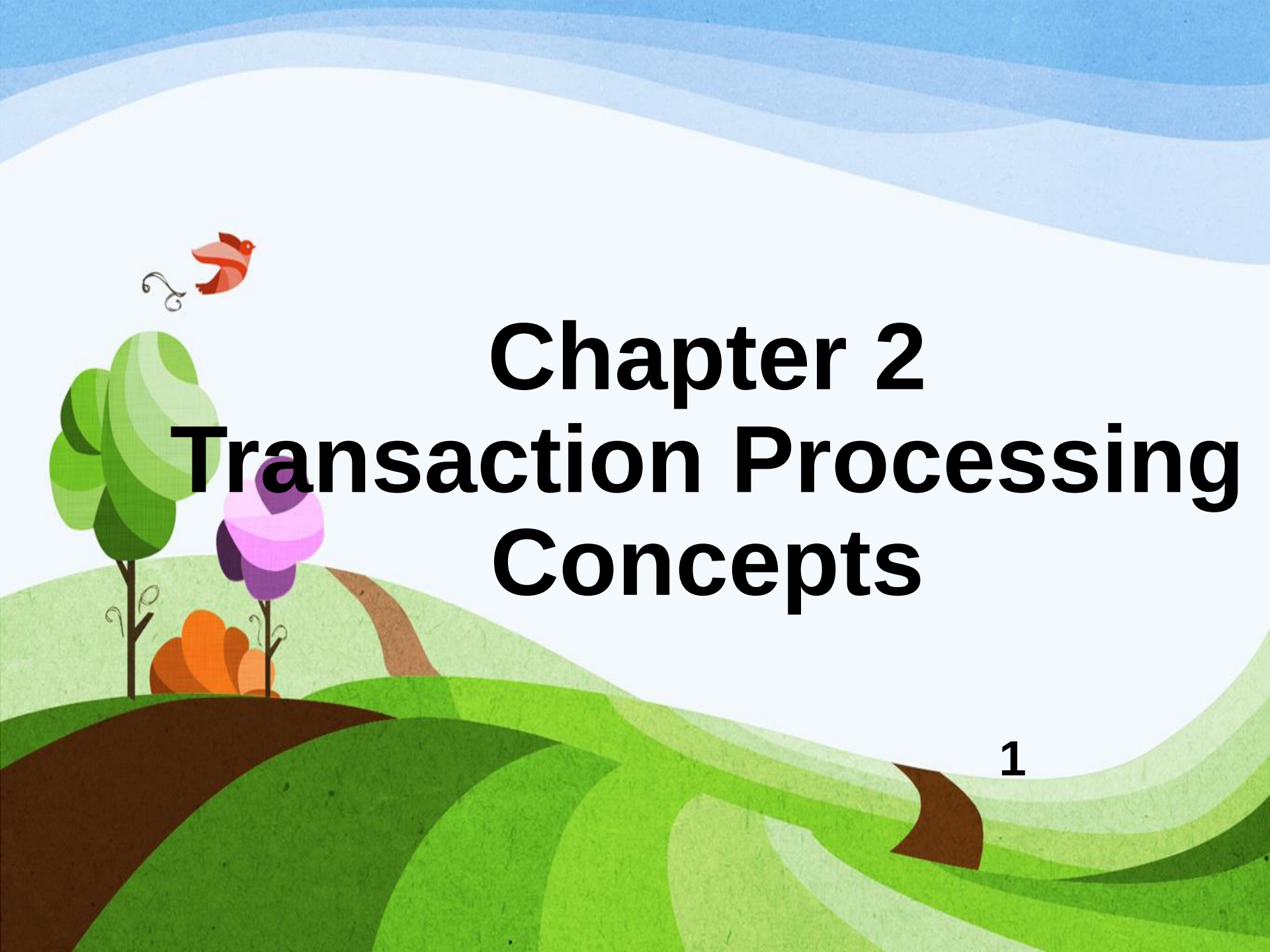




Chapter 2

Transaction Processing

Concepts

Chapter Outline

- Transaction Concepts
- Transaction States
- Properties of Transaction
- Schedules of Transactions
- Conflicting Operations in Schedules
- Serializability of Schedules
- Transaction Isolation Levels
- Transaction Support in SQL Server

Transaction Concepts

- **Transaction** is an action, or series of actions, performed against the contents of the database.
- **Transaction** is an atomic unit of operation that is either completed in its entirety or not done at all.
- Transaction operations may include:
 - Read operations (SELECT).
 - Write operations (INSERT, UPDATE, DELETE).
 - Transaction boundaries
 - Commit transaction
 - Rollback (abort) transaction

Transaction States

- Transaction states may be:
 - Active state (executing read, write operations).
 - Partially committed state (ended but waiting for system checks to determine success or failure).
 - Committed state (transaction succeeded).
 - Failed state (transaction failed, must be rolled back).
 - Terminated State (transaction leaves system).
- State transition diagram illustrating the states for transaction execution is given below:

Transaction States (Cont'd...)

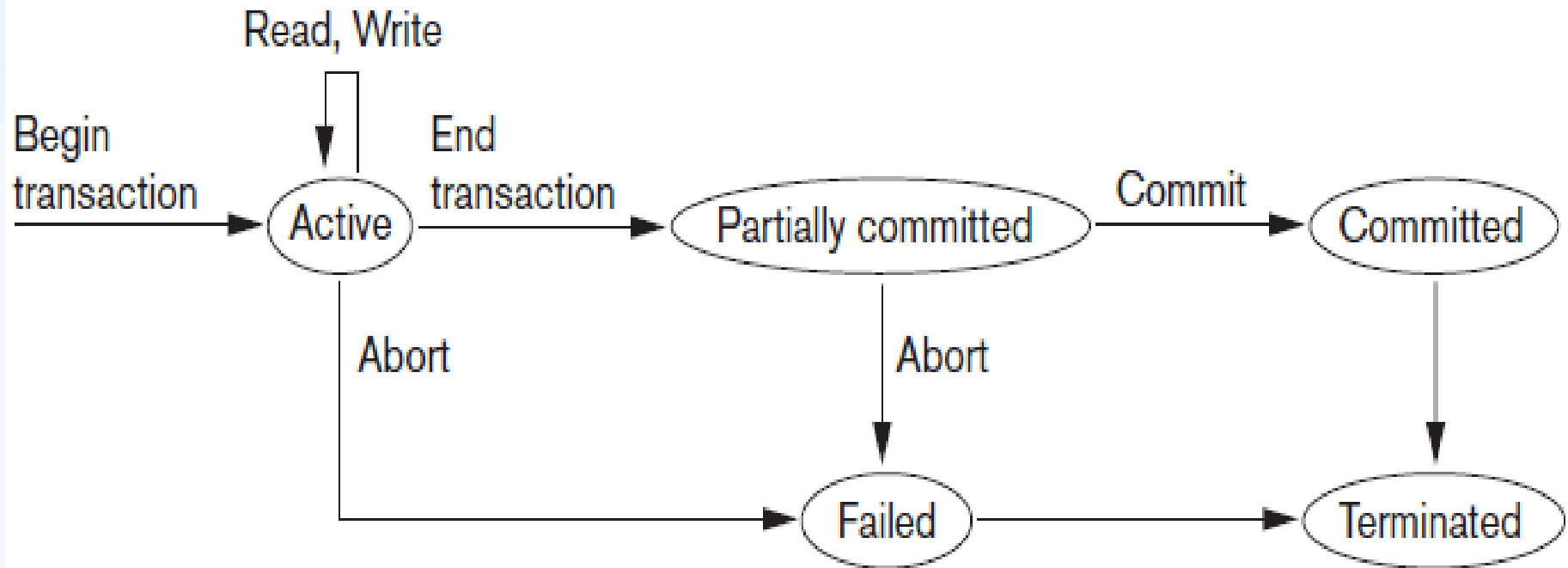


Figure 2.1: State transition diagram illustrating the states for transaction execution

Properties of Transaction

➤ Called ACID properties:

- **Atomicity:** A transaction is either performed in its entirety or not performed at all(**all-or-none**).
- **Consistency:** A transaction must transform the database from one consistent state to another consistent state.
- **Isolation:** Transactions execute independently of one another.
- **Durability or permanency:** Once a transaction is committed, it is permanent and its changes applied to the database must never be lost.

Schedules of Transactions

- A **schedule** S of n transactions $T_1, T_2, \dots, T_n$ is an ordering of the operations of the transactions.
- The order of operations in S is considered to be a total ordering, meaning that for any two operations in the schedule, one must occur before the other.

Figure 2.2: The schedule in this figure can be written using the notation as:
 $S: r_1(X); r_2(X); w_1(X);$
 $r_1(Y); w_2(X); w_1(Y);$

	T_1	T_2
Time ↓	Read_Item(X)	Read_Item(X)
	Write_Item(X)	
	Read_Item(Y)	Write_Item(X)
	Write_Item(Y)	

Conflicting Operations

- Two operations in a schedule are said to **conflict** if they satisfy all of the following three conditions:
- (1) they belong to different transactions.
 - (2) they access the same item X .
 - (3) at least one of the operations is a write operation.

For example the following operations are conflict.

- ✓ $r_1(X)$ and $w_2(X)$
- ✓ $r_2(X)$ and $w_1(X)$
- ✓ $w_1(X)$ and $w_2(X)$

Non-Conflicting Operations

- However, the following operations do not conflict in a schedule:
 - ✓ $w_2(X)$ and $w_1(Y)$, because they operate on distinct data items X and Y .
 - ✓ $r_1(X)$ and $w_1(X)$, because they belong to the same transaction.
 - ✓ $r_1(X)$ and $r_2(X)$, since they are both read operations.

Serializability of Schedules

- **Serial schedule** is a schedule where the operations of each transaction are executed consecutively without any interleaved operations from other transactions.
- If there are interleaved operations in a schedule, the schedule is called **nonserial schedule**.
- **Non-serial schedule** may produce an erroneous result because of the lost update problem.
- In a serial schedule, only one transaction at a time is active -> the commit (or abort) of the active transaction initiates the next transaction.

Serializability of Schedules...

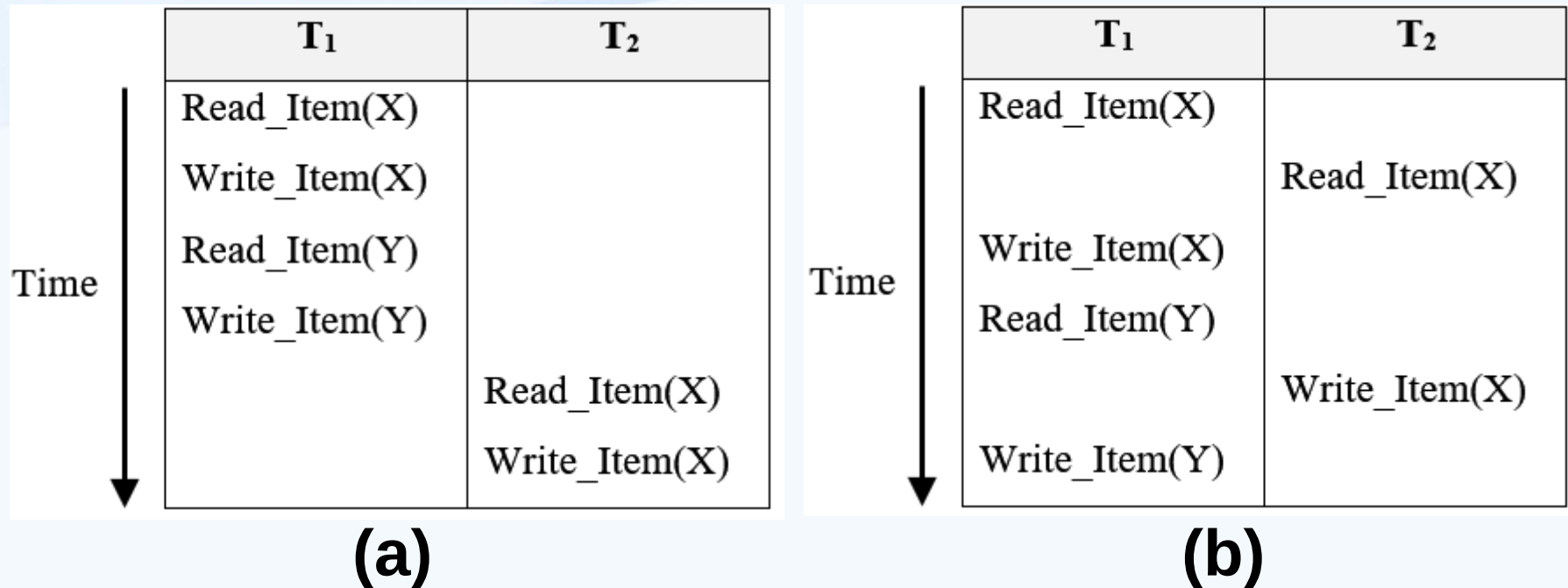


Figure 2.3: For two transactions T_1 and T_2

- (a) Serial schedule: T_1 followed by T_2
- (b) Non serial schedule with interleaving of operations

Transaction Isolation Levels

- The isolation levels in SQL Server are: Read uncommitted, Read committed, Repeatable read and Serializable.
- **Read uncommitted:** Allows uncommitted data to be read.
 - ✓ It is the lowest isolation level allowed by SQL.
 - ✓ Doesn't hold locks when reading data.
- **Read committed:** Allows only committed data to be read.
 - ✓ It is the default isolation level for SQL Server.
 - ✓ Holds lock

Transaction Isolation Levels...

- **Repeatable read:** Allows only committed data to be read and further requires that, between two reads of a data item by a transaction, no other transaction is allowed to update it.
 - ✓ Holds lock for the duration of a transaction to prevent other transactions modifying the data.
- **Serializable:** Usually ensures serializable execution.
 - ✓ Holds lock for transaction duration.
 - ✓ The most isolated level and damaging to concurrency.

Exercise

1. What is the difference between Read Committed and Repeatable Read isolation levels?
2. What is the difference between Repeatable Read and Serializable isolation levels?

Implementation of Isolation Levels

- We know that many database systems run, by default, at the read-committed isolation level.
- In SQL, it is possible to change the isolation level explicitly using the syntax below:

Set Transaction Isolation Level

**Serializable | Repeatable Read | Read
Committed | Read Uncommitted;**

- The above syntax is supported by Oracle, PostgreSQL and SQL Server.
- Changing of the isolation level must be done as the first statement of a transaction.

Transaction in SQL Server

- Begin transaction syntax:

BEGIN TRAN

- Commit transaction syntax:

COMMIT TRAN

- ✓ Indicates successful end of transaction.
- ✓ At the point you issue commit, transaction is considered to be durable and can't undo.

Transaction in SQL Server...

- Rollback transaction syntax:

ROLLBACK TRAN

- ✓ Indicates end of transaction with failure.
- ✓ Undo to the beginning of the transaction.
- ✓ Rollback is the preferred way to recover a database after a transaction in progress terminates abnormally.

Structured Exception Handling

BEGIN TRY

BEGIN TRANSACTION

--T-SQL (INSERT, UPDATE, DELETE, CREATE, ...)

COMMIT TRANSACTION

END TRY

BEGIN CATCH

SELECT ERROR_NUMBER() AS ErrorNumber,
ERROR_MESSAGE() AS ErrorMessage

ROLLBACK TRANSACTION

END CATCH